Estructuras de Datos, 2025-2

Para entregar el domingo 7 de septiembre de 2025
A las 23:59 en la arena de programación

Instrucciones para la entrega

- Para esta tarea y todas las tareas futuras en la arena de programación, la entrega de soluciones es *individual*. Por favor escriba claramente su nombre, código de estudiante y sección en el inicio de cada archivo de código (a modo de comentario). Adicionalmente, cite cualquier fuente de información que utilizó. Los códigos fuente que suba a la arena de programación deben der de su completa autoría.
- En cada problema debe leer los datos de entrada de la forma en la que se indica en el enunciado y debe imprimir los resultados con el formato allí indicado. No debe agregar mensajes ni agregar o eliminar datos en el proceso de lectura. La omisión de esta indicación puede generar que su programa no sea aceptado en la arena de programación.
- Aunque la arena de programación soporta envíos en otros lenguajes de progamación como Python, **todos los problemas de esta tarea deben ser resueltos en C++**. Los envíos en Python no serán admitidos.
- Debe enviar sus soluciones a través de la arena. Antes de subir sus soluciones asegurese de realizar pruebas con los casos de pruebas proporcionados para verificar que el programa finalice y no se quede en un ciclo infinito.
- El primer criterío de evaluación será la aceptación del problema en la arena cumpliendo los requisitos indicados en los enunciados de los ejercicios y en este documento. Además, también se considerará la complejidad de las soluciones planteadas y se tendrán en cuenta aspectos de estilo como no usar `break` ni `continue` y que las funciones deben tener únicamente una instrucción `return` que debe estar en la última línea.
- En cada archivo **debe indicar en el bloque de comentarios del encabezado la complejidad de la solución y su correspondiente explicación**.

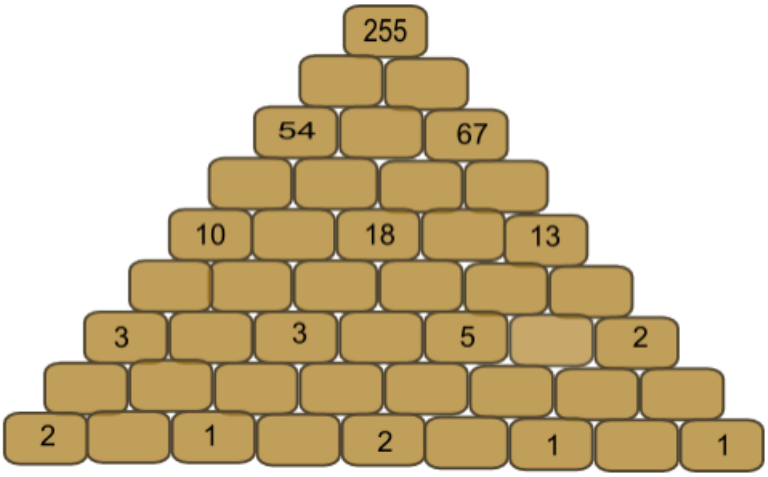
Problemas prácticos

Hay cinco problemas prácticos cuyos enunciados aparecen a partir de la siguiente página.

A - Problem A

Source file name: `bricks.cpp`
Time limit: 1 second

This in not "*another brick in the wall*", it's just a matter of adding numbers. Suppose you have a wall with the shape of a triangle, like the one shown below. The wall has 9 rows and row *i* has exactly *i* bricks, considering that top row is the first one and bottom row is the ninth. Some bricks are labeled with a number and other ones are blank. Notice that labeled bricks appear only on odd rows and they occupy odd positions within the row. The problem you must solve is finding a suitable number for each blank brick taking into account one simple rule: the number of a brick is obtained by adding the numbers of the two bricks below it. Obviously, this rule does not apply to the ninth row. Numbers are supposed to be integers.



Input

The first line of the input contains an integer *N*, indicating the number of test cases. This line is followed by the lines corresponding to the test cases. Each test case is described in five lines. These five lines correspond to odd rows of the wall, from top to bottom, as described above. Line *i* contains the numbers corresponding to odd bricks on row *i* of the wall (that is, non blank bricks), enumerated from left to right and separated with a single space. It is supposed that each test case is correct, that is, there exists a solution to the problem that the case describes.

The input must be read from standard input.

Output

For each test case, the output should consist of nine lines describing the numbers of all bricks of the wall. So, line *i* should contain the numbers corresponding to the *i* bricks on row *i* of the wall, enumerated from left to right and separated by a single space.

Note: Here we have an example with two test cases. The first one corresponds to the wall depicted above.

The output must be written to standard output.

Sample Input	**Sample Output**
2 255 54 67 10 18 13 3 3 5 2 2 1 2 1 1 256 64 64 16 16 16 4 4 4 4 1 1 1 1 1	255 121 134 54 67 67 23 31 36 31 10 13 18 18 13 5 5 8 10 8 5 3 2 3 5 5 3 2 2 1 1 2 3 2 1 1 2 0 1 0 2 1 1 0 1 256 128 128 64 64 64 32 32 32 32 16 16 16 16 16 8 8 8 8 8 8 4 4 4 4 4 4 4 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1

B - Problem B

Source file name: `dahdahdah.cpp`
Time limit: 1 second

Morse code is a method for long-distance transmission of textual information without using the usual symbols. Instead information is represented with a simpler, binary, alphabet composed of short and long beeps. The short beep is called dih, and the long beep is called dah. For instance, the code for the letter O is dah dah dah (three long beeps). Actually, because the codification is not prefix-free, there is also a third symbol, which is silence. The code between two letters is a simple silence, the code between two words is a double silence.

You have been assigned the job to translate a message in Morse code. The signal has already been digitalized in the following fashion: dih is represented by a dot (`.`), dah is represented by a dash (`-`). Simple and double silences are represented by a single space character and two space characters respectively.

The following table represents the Morse code of all the characters that your program need to be able to handle.

Symbol	Code	Symbol	Code	Symbol	Code	Symbol	Code	Symbol	Code	Symbol	Code
A	.-	J	.---	S	...	1	.----	.	.-.-.-	:	---...
B	-...	K	-.-	T	-	2	..---	,	--..--	;	-.-.-.
C	-.-.	L	.-..	U	..-	3	...--	?	..--..	=	-...-
D	-..	M	--	V	...-	4	....-	'	.----.	+	.-.-.
E	.	N	-.	W	.--	5	.....	!	-.-.--	-	-....-
F	..-.	O	---	X	-..-	6	-....	/	-..-.	_	..--.-
G	--.	P	.--.	Y	-.--	7	--...	(	-.-.	"	.-..-.
H	....	Q	--.-	Z	--..	8	---..	)	-.--.-	@	.--.-.
I	..	R	.-.	0	-----	9	----.	&	.-...		

Input

The first line of input gives the number of cases T. T test cases follow. Each one is a sequence of dot, dash and space characters. Two messages are separated by a newline. The maximum length of a message is 2000.

The input must be read from standard input.

Output

The output is comprised of one paragraph for each message. The paragraph corresponding to the n-th message starts with the header `'Message #n'`, on a line on its own. Each decoded sentence of the message appears then successively on a line of its own. Two paragraphs are separated by a blank line.

The sentences shall be printed in uppercase.

The output must be written to standard output.

Sample Input

```
2
... --- ...
.--- --- -...  -.. --- -. .  ..--..  ..-. .. -. . -.-.--
```

Sample Output

```
Message #1
SOS

Message #2
JOB DONE ? FINE!
```

C - Problem C

Source file name: `die.cpp`
Time limit: 1 second

Life is not easy. Sometimes it is beyond your control. Now, as contestants of ACM ICPC, you might be just tasting the bitter of life. But don't worry! Do not look only on the dark side of life, but look also on the bright side. Life may be an enjoyable game of chance, like throwing dice. Do or die! Then, at last, you might be able to find the route to victory.

This problem comes from a game using a die. By the way, do you know a die? It has nothing to do with "death". A die is a cubic object with six faces, each of which represents a different number from one to six and is marked with the corresponding number of spots. Since it is usually used in pair, "a die" is a rarely used word. You might have heard a famous phrase "the die is cast", though.

When a game starts, a die stands still on a flat table. During the game, the die is tumbled in all directions by the dealer. You will win the game if you can predict the number seen on the top face at the time when the die stops tumbling.

Now you are requested to write a program that simulates the rolling of a die. For simplicity, we assume that the die neither slips nor jumps but just rolls on the table in four directions, that is, north, east, south, and west. At the beginning of every game, the dealer puts the die at the center of the table and adjusts its direction so that the numbers one, two, and three are seen on the top, north, and west faces, respectively. For the other three faces, we do not explicitly specify anything but tell you the golden rule: the sum of the numbers on any pair of opposite faces is always seven.

Your program should accept a sequence of commands, each of which is either "north", "east", "south", or "west". A "north" command tumbles the die down to north, that is, the top face becomes the new north, the north becomes the new bottom, and so on. More precisely, the die is rotated around its north bottom edge to the north direction and the rotation angle is 90 degrees. Other commands also tumble the die accordingly to their own directions. Your program should calculate the number finally shown on the top after performing the commands in the sequence. Note that the table is sufficiently large and the die never falls off during the game

Input

The input consists of one or more command sequences, each of which corresponds to a single game. The first line of a command sequence contains a positive integer, representing the number of the following command lines in the sequence. You may assume that this number is less than or equal to 10000. A line containing a zero indicates the end of the input. Each command line includes a command that is one of `'north'`, `'east'`, `'south'`, and `'west'`. You may assume that no white space occurs in any lines.

The input must be read from standard input.

Output

For each command sequence, output one line containing solely the number on the top face at the time when the game is finished.

The output must be written to standard output.

Sample Input	**Sample Output**
1 north 3 north east south 0	5 1

D - Problem D

Source file name: `odd.cpp`
Time limit: 1 second

You have an array $a_1, a_2, \ldots, a_n$. Answer q queries of the following form: If we change all elements in the range $a_l, a_{l+1}, \ldots, a_r$ of the array to k, will the sum of the entire array be odd?

Note that queries are **independent** and do not affect future queries.

Input

Each test contains multiple test cases. The first line contains the number of test cases t. The description of the test cases follows.

The first line of each test case consists of 2 integers n and q ($1 \le n \le 2 \cdot 10^5$; $1 \le q \le 2 \cdot 10^5$) — the length of the array and the number of queries. The second line of each test case consists of n integers a_i ($1 \le a_i \le 10^9$) — the array a. The next q lines of each test case consists of 3 integers l, r, k ($1 \le l \le r \le n$; $1 \le k \le 10^9$) — the queries.

The input must be read from standard input.

Output

For each query, output `"YES"` if the sum of the entire array becomes odd, and `"NO"` otherwise.

Note: For the first test case:

- If the elements in the range $(2, 3)$ would get set to 3 the array would become $\{2, 3, 3, 3, 2\}$, the sum would be $2 + 3 + 3 + 3 + 2 = 13$ which is odd, so the answer is `"YES"`.
- If the elements in the range $(2, 3)$ would get set to 4 the array would become $\{2, 4, 4, 3, 2\}$, the sum would be $2 + 4 + 4 + 3 + 2 = 15$ which is odd, so the answer is `"YES"`.
- If the elements in the range $(1, 5)$ would get set to 5 the array would become $\{5, 5, 5, 5, 5\}$, the sum would be $5 + 5 + 5 + 5 + 5 = 25$ which is odd, so the answer is `"YES"`.
- If the elements in the range $(1, 4)$ would get set to 9 the array would become $\{9, 9, 9, 9, 2\}$, the sum would be $9 + 9 + 9 + 9 + 2 = 38$ which is even, so the answer is `"NO"`.
- If the elements in the range $(2, 4)$ would get set to 3 the array would become $\{2, 3, 3, 3, 2\}$, the sum would be $2 + 3 + 3 + 3 + 2 = 13$ which is odd, so the answer is `"YES"`.

The output must be written to standard output.

Sample Input	**Sample Output**
2 5 5 2 2 1 3 2 2 3 3 2 3 4 1 5 5 1 4 9 2 4 3 10 5 1 1 1 1 1 1 1 1 1 1 3 8 13 2 5 10 3 8 10 1 10 2 1 9 100	YES YES YES NO YES NO NO NO NO YES

E - Problem E

Source file name: `poker.cpp`
Time limit: 1 second

Poker is one of the most widely played card games, and King's Poker is one of its variations. The game is played with a normal deck of 52 cards. Each card has one of 4 suits and one of 13 ranks. However, in King's Poker card suits are not relevant, while ranks are Ace (rank 1), 2, 3, 4, 5, 6, 7, 8, 9, 10, Jack (rank 11), Queen (rank 12) and King (rank 13). The name of the game comes from the fact that in King's Poker, the King is the highest ranked card. But this is not the only difference between regular Poker and King's Poker. Players of King's Poker are dealt a hand of just three cards. There are three types of hands:

- A *set*, made of three cards of the same rank.
- A *pair*, which contains two cards of the same rank, with the other card unmatched.
- A *no-pair*, where no two cards have the same rank.

Hands are ranked using the following rules:

- Any set defeats any pair and any no-pair.
- Any pair defeats any no-pair.
- A set formed with higher ranked cards defeats any set formed with lower ranked cards.
- If the matched cards of two pairs have different ranks, then the pair with the higher ranked matched cards defeats the pair with the lower ranked matched cards.
- If the matched cards of two pairs have the same rank, then the unmatched card of both hands are compared; the pair with the higher ranked unmatched card defeats the pair with the lower ranked unmatched card, unless both unmatched cards have the same rank, in which case there is a tie.

A new software house wants to offer King's Poker games in its on-line playing site, and needs a piece of software that, given a hand of King's Poker, determines the set or pair with the lowest rank that beats the given hand. Can you code it?

Input

Each test case is described using a single line. The line contains three integers A, B, and C representing the ranks of the cards dealt in a hand ($1 \leq A, B, C \leq 13$).

The last test case is followed by a line containing three zeros.

The input must be read from standard input.

Output

For each test case output a single line. If there exists a set or a pair that beats the given hand, write the lowest ranked such a hand. The beating hand must be written by specifying the ranks of their cards, in non-decreasing order. If no set or pair beats the given hand, write the character `'*'` (asterisk).

The output must be written to standard output.

Sample Input	**Sample Output**
1 1 1 1 1 12 1 1 13 1 13 1 10 13 10 1 2 2 13 13 13 13 12 13 12 12 12 3 1 4 1 5 9 0 0 0	2 2 2 1 1 13 1 2 2 1 2 2 1 11 11 2 2 3 * 1 1 1 13 13 13 1 1 2 1 1 2